2020 IEEE-RAS 20th International Conference on Humanoid Robots (Humanoids)
July 19-21, 2021. Munich, Germany

2020 IEEE-RAS 20th International Conference on Humanoid Robots (Humanoids) | 978-1-7281-9372-4/21/$31.00 ©2021 IEEE | DOI: 10.1109/HUMANOIDS47582.2021.9555680

Safe Data-Driven Contact-Rich Manipulation

Ioanna Mitsioni*, Pouria Tajvar*, Danica Kragic, Jana Tumova, and Christian Pek

***Abstract*— In this paper, we address the safety of data-driven control for contact-rich manipulation. We propose to restrict the controller's action space to keep the system in a set of safe states. In the absence of an analytical model, we show how Gaussian Processes (GP) can be used to approximate safe sets. We disable inputs for which the predicted states are likely to be unsafe using the GP. Furthermore, we show how locally designed feedback controllers can be used to improve the execution precision in the presence of modelling errors. We demonstrate the benefits of our method on a pushing task with a variety of dynamics, by using known and unknown surfaces and different object loads. Our results illustrate that the proposed approach significantly improves the performance and safety of the baseline controller.**

I. INTRODUCTION

Learning-based data-driven control (DDC) techniques have proven to be successful in tackling contact-rich manipulation tasks [1]. These tasks are particularly challenging, since interactions with the environment are usually difficult to model and their dynamics change rapidly. DDC approaches learn to approximate the dynamics of such tasks, e.g., using neural networks, from a given data-set of trajectories [2] or through random exploration [3] (see Fig. 1). Subsequently, the learned models are combined with receding horizon control techniques, like model predictive control (MPC), to produce trajectories for the system. Receding horizon techniques have the advantage that modelling errors will not accumulate, as they re-sample the real system. In addition, once a model of the system has been acquired, different desired behaviors can be encoded through the cost function, without retraining the model. However, in their basic form, data-driven controllers do not focus on avoiding unsafe states or safeguarding against residual modelling errors that may impede the task execution.

Developing safe data-driven controller for contact-rich tasks requires addressing the safety properties of the controller and the model disturbances that may occur throughout the execution. First, the controller should always keep the system within sets of safe states. This constraint is of paramount importance during contact and cannot be easily encompassed by a cost function. Second, the controller needs to actively correct the inputs based on the error between predictions of the learned model and observed states. Re-sampling the real system can avoid diverging predictions to some degree, but the varying contact dynamics can cause the execution to deviate from the prediction in-between samples and potentially violate the safety constraints. To counteract this divergence, the control inputs need to be actively refined, i.e., adjusted, to lower the error between prediction and actual execution. As a side-effect, this can also increase the task execution efficiency.

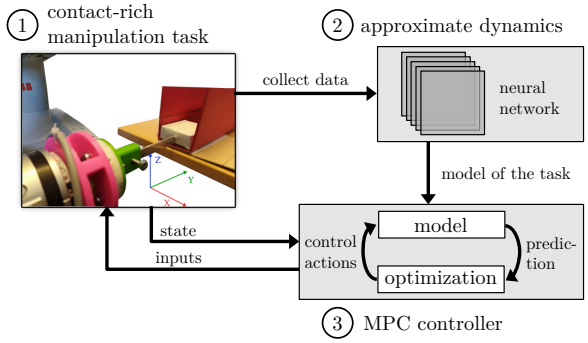


Fig. 1: Overview of data-driven control for contact-rich manipulation tasks. 1) Execution data of a contact-rich manipulation task are collected, e.g., pushing an object on a variety of surfaces. 2) Based on the collected data samples, the dynamics of the task are approximated, e.g., using a neural network. 3) The learned model is used in an MPC framework to solve the contact-rich manipulation task.

To attain the safety properties, we aim at constraining the DDC to keep the system in sets of safe states. However, the computation of such sets usually requires an analytical model of the system and refinement of the control inputs to ensure that the transitions between states will remain in the sets. In this work, we address these two challenges by approximating the safe sets in the absence of an analytical model and by synthesizing a set of local feedback motion primitives to reduce tracking errors. We demonstrate our approach on a one-dimensional pushing task, which is a well-studied problem in robotics [4]–[6]. During pushing trials, it is easy to capture a variety of dynamics by changing the surfaces and the loads of the pushed object. In addition, its dimensionality makes it an excellent test-bed for clearly observing the effect of our approach on DDC.

A. Related Work

To avoid the inherent risks of contact-rich manipulation for the robot and its environment, various approaches have been proposed that can mainly be classified in three categories: designing fail-safe mechanisms [7], [8], incorporating safety criteria in the reward/cost function [9], [10] and lastly, limiting the permitted actions [11], [12]. Restricting the actions

* The authors contributed equally to this work.

All of the authors are with the Division of Robotics, Perception and Learning, School of Electrical Engineering and Computer Science, KTH Royal Institute of Technology, 114 28 Stockholm, Sweden. Mail addresses: {mitsioni, tajvar, dani, tumova, pek2}@kth.se

This work was supported by the Foundation for Strategic Research, Swedish Research Council, European Research Council and Knut and Alice Wallenberg Foundation. The authors are affiliated with Digital Futures.

has the advantage of enabling an explicit expression of unsafe actions, while minimally intervening with the nominal function of the controller. Limiting the input space also improves computational efficiency due to the smaller action space [13]. In general, limiting the actions for safety can be implemented as an adjustment after selection of an action [7], or by pre-filtering actions, [12]. Our safety approach pre-filters unsafe actions; however, instead of a fixed safety criterion, we employ a classifier that allows using learning-based safety evaluation, as well as explicit safety functions.

Our pre-filter checks whether the system remains in a set of safe states after executing a chosen action. Control barrier functions (CBFs) are a popular technique to compute sets of safe states [14]. CBFs partition the state space of the system into sets of safe and unsafe states. Then, safety can be shown by proving that the safe set is forward invariant w.r.t. the system dynamics, and that there will always be a control action that prevents the system from crossing the CBF, e.g., demonstrated in [15]–[17].

Similarly, invariant sets (IS) contain states which allow the system to remain safe [18]–[21], i.e., there exists at least one safe action for each state within an IS. Unfortunately, CBFs and IS generally require an analytical model of the system to compute the set of safe states. We show that sets of safe states for contact-rich manipulation tasks can be approximated by Gaussian Processes (GPs) [22], [23]. GPs have successfully been applied to learn sets of safe controller parameters [24] or sets of safe end-effector positions for grasping [25].

To ensure that the system follows safe actions during execution, we use feedback motion primitives to keep the state close to the prescribed states between sampling times. This idea has been explored for manipulators using Lyapunov stability analysis [26] and using disturbance observers for free-space motion [27]. These approaches, however, are not directly extensible to motions with contact due to lack of an a-priori defined model for Lyapunov function or disturbance observer construction. Instead, we propose feedback synthesis through local linearizations of the learned model and construction of a library of feedback motion primitives. Data-driven local linearization of the dynamics has been explored in [28] for manipulator free space motion and feedback motion primitives have been previously studied in various works for mobile robots [29]–[31]. As opposed to mobile robots where the dynamics remain largely the same in the entire state space, for contact-rich manipulation this does not necessarily hold. We address this problem by constructing local primitives instead of global ones.

Our previous work [32] focused on demonstrating the performance of DDC for contact-rich manipulation tasks. Here, we show how the safety of DDC frameworks can be improved without impeding performance. To that end, we present how

1) sets of safe states can be accurately approximated in the absence of analytical models using Gaussian processes,
2) feedback motion primitives can be synthesized based on the learned model, and
3) the system can be kept in the generated safe sets in the presence of modelling errors.

We demonstrate the benefits of our approach on a one-dimensional pushing task, in which a robot is pushing an object with different loads on known/unknown surfaces. Our results indicate that our approach significantly improves both the safety and efficiency, in terms of task completion time and average progress, of the DDC baseline approach [32].

II. PRELIMINARIES

A. Data-Driven Control

We denote the end-effector position of our robot as $\mathbf{x}^P \in \mathbb{R}^3$ and the external forces measured through a force sensor as $\mathbf{x}^F \in \mathbb{R}^3$. The system control inputs are reference forces along the axes of the end-effector, given by $\mathbf{u} \in \mathbb{R}^3$. The task dynamics are modelled using the state $\mathbf{x} = [\Delta\mathbf{x}^P, \mathbf{x}^F]^T$, where $\Delta\mathbf{x}^P$, $\mathbf{x}^F$ are relative displacements and external forces respectively. By considering relative displacements, the model is position-invariant and thus, valid regardless of where in space the task is executed.

Additionally, we define the state over sequences of $M \in \mathbb{N}_+$ non-overlapping measurements, called blocks. Every block b contains the timesteps from $t_b = bM$ to $t_{b+1} = (b+1)M - 1$. Thus, the block-dependent state and inputs are given by $\mathbf{x}_b = [\Delta\mathbf{x}_b^P, \mathbf{x}_b^F]^T \in \mathbb{R}^{M\times 6}$ and $\mathbf{u}_b \in \mathbb{R}^{M\times 3}$, respectively. The computation of the relative displacements is done by subtracting the past block's last position from every position in the current one

$$\boldsymbol{\Delta}\mathbf{x}_b^P = \mathbf{x}_b^{P,M} - \mathbf{1}^M \mathbf{x}_{(b-1)M-1}^P, \tag{1}$$

where $\mathbf{x}_b^{P,M}$ and $\mathbf{1}^M$ denote the vector of the positions of the block and an all-ones vector of length M.

In DDC, we aim to learn the dynamics model f of the task that can be used in a model predictive control (MPC) framework:

$$\mathbf{x}_{b+1} = f(\mathbf{x}_b, \mathbf{u}_b), \tag{2}$$

where $\mathbf{x}_{b+1}$ denotes the predicted subsequent state of the system when starting in state $\mathbf{x}_b$ and applying control input $\mathbf{u}_b$. Given the learned model f, the MPC framework determines the optimal control input $\mathbf{u}^*_{b:(H_b-1)}$ over a horizon of blocks $H_b \in \mathbb{N}_+$ by minimizing a cost function C of the states and inputs:

$$\begin{aligned} \mathbf{u}^*_{b:(H_b-1)} = \arg\min_{\mathbf{u}} & \sum_{k=0}^{H_b-1} C\big(\mathbf{x}_{b+k+1}, \mathbf{u}_{b+k}\big), \\ \text{subject to } & \mathbf{x}_{b+k+1} = f(\mathbf{x}_{b+k}, \mathbf{u}_{b+k}), \\ & \mathbf{x}_b = \mathbf{x}(t_b) \end{aligned} \tag{3}$$

B. Sets of Safe States

To increase the safety of the MPC formulation in (3), we constrain the controller to keep the system within a set of safe states. Safety for contact-rich manipulation tasks can refer to any measure-able observation, such as avoiding unwanted contacts and joint limits, or keeping external forces low. Given a desired safety property for the task, the state-space of our system can be partitioned into a set of safe and

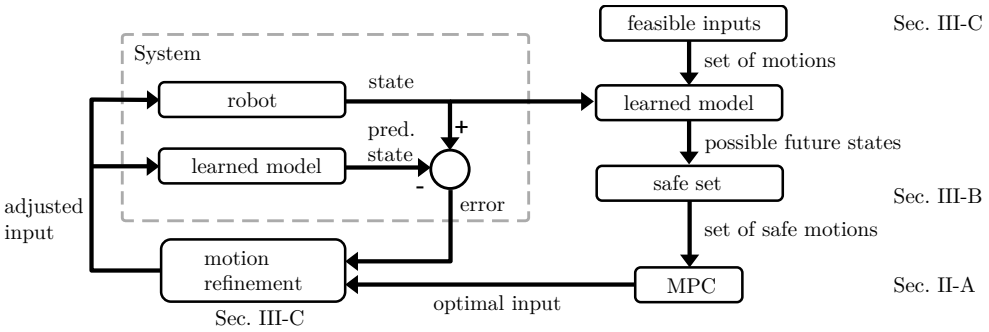


Fig. 2: Overview of the proposed approach.Using the approximated safe set (see Sec. III-B), the MPC will only consider safe feasible inputs. The optimal input of the MPC is adjusted (see Sec. III-C) based on the error between the predicted and actual state.

unsafe states. We use the labels $\top$ and $\bot$ to denote whether a given state $\mathbf{x}_b$ is safe or unsafe, respectively. The function $\Psi : \mathbb{R}^{M\times 6} \rightarrow \{\top, \bot\}$ provides the binary safety label for a given state. Inspired by the definition of invariant sets (see Sec. I-A), we define sets of safe states as:

Definition 1 (Safe Set $\mathcal{S}$)
A set $\mathcal{S} \subseteq \mathbb{R}^{M\times 6}$ is a safe set if $\forall \mathbf{x} \in \mathcal{S} : \Psi(\mathbf{x}) = \top \wedge \exists \mathbf{u} : f(\mathbf{x}, \mathbf{u}) \in \mathcal{S}$.

Intuitively, safe sets $\mathcal{S}$ only contain safe states that allow our system to *remain* safe. The computation of $\mathcal{S}$ usually requires an analytical model.

III. METHOD

A. Overview

Fig. 2 presents an overview of the proposed approach. We start with the current state $\mathbf{x}_b$ of the robot. This state is an input to the model of the system dynamics together with the a set of feasible inputs $\mathbf{U}_b^{ff}$. As a result of this step, we obtain a set of possible future states $\mathbf{x}_{b+1}$ of the system.

Subsequently, we check which of the possible future states is enclosed in a set of safe states of our system. This set is approximated using a Gaussian Process Classifier, or GPC (see Sec. III-B). If a state is enclosed in the safe set, we consider the corresponding feasible input as safe. Ultimately, this step identifies the set of safe inputs $\mathbf{U}_{b,\text{safe}}^{ff}$. The set of safe inputs is then used in the MPC framework to obtain the optimal input $\mathbf{u}_{b+1}^*$ as determined through the optimization problem in Eq. (3) for a horizon of $H_b = 1$.

Lastly, we refine the motion by applying a feedback motion primitive that corresponds to the selected safe input $\mathbf{u}_{b+1}^*$, to account for possible modelling errors during execution. The motion primitives are constructed for different regions of the state space and for each input from $\mathbf{U}_b^{ff}$, using local linearizations of the dynamics (see Sec. III-C). Based on the error $\mathbf{e}_b$ between the prediction (i.e. prescribed state) $\mathbf{x}_{b-1}$ for the previous cycle and the actual position $\mathbf{x}_b$ we resulted in, the optimal input $\mathbf{u}_{b+1}^*$ is refined to ensure the robot will reach the safe goal state by including the feedback rule of the motion primitive. The adjusted input $\tilde{\mathbf{u}}_{b+1}^*$ is then sent to the robot. The presented approach repeats at every control cycle of the system.

B. Approximation of Safe Sets

The computation of safe sets $\mathcal{S}$ (see Def. 1) generally requires an analytical model of the system. However, since we do not have an analytical expression of our model f, we approximate the safe set $\mathcal{S}$ using examples of safe and unsafe states from previous executions of the system. Without loss of generality, we assume that a labeled dataset $\mathcal{D}$ is provided that contains tuples $(\mathbf{x}_b, \psi, \mathbf{x}_{b+1})$, where if $\mathbf{x}_b$ is the state of the robot at time t_b, and $\mathbf{x}_{b+1} = f(\mathbf{x}_b, \mathbf{u}_b), \mathbf{u}_b \in \mathcal{U}$, is the subsequent state, $\psi \in \{\top, \bot\}$ is the binary label denoting whether the desired property holds for state $\mathbf{x}_{b+1}$, i.e., $\Psi(\mathbf{x}_{b+1}) = \psi$

Given the labeled dataset $\mathcal{D}$, we approximate the boundary $\partial\mathcal{S}$ of $\mathcal{S}$ using Gaussian Processes (GPs). The use of GPs has the advantage that we can capture the inherent uncertainty about the exact safe set [23]. The boundary $\partial\mathcal{S}$ is partitioning our state space into safe and unsafe states. Thus, approximating $\mathcal{S}$ is a classification problem of deciding whether a state is enclosed in $\mathcal{S}$. The decision boundary of the classifier corresponds to the boundary $\partial\mathcal{S}$ of the set of safe states (see Fig. 3). For classification purposes, GPs with a logistic link function have shown great accuracy. Interested readers are referred to [23, Chap. 3] for more details.

To train the GP classifier, we split the dataset into examples of safe $\mathcal{D}_{\text{safe}}$ and unsafe states $\mathcal{D}_{\text{unsafe}}$ according to Def. 1, i.e., a state is safe if we can determine an input to another safe state:

$$\begin{aligned} \mathcal{D}_{\text{safe}} &:= \{\mathbf{x}_b \,|\, (\mathbf{x}_b, \psi, \mathbf{x}_{b+1}) \in \mathcal{D} \wedge \psi = \top\}, \\ \mathcal{D}_{\text{unsafe}} &:= \{\mathbf{x}_b \,|\, (\mathbf{x}_b, \psi, \mathbf{x}_{b+1}) \in \mathcal{D} \wedge \psi = \bot\}, \end{aligned} \tag{4}$$

where $\psi = \Psi(\mathbf{x}_{b+1})$. During training, the non-Gaussian posterior of the GP classifier is approximated using a Laplace approximation in the case of a binary label dataset [23, Chap. 3]. The resulting Gaussian allows us to provide probabilistic estimates of the safe set boundary.

C. Feedback Motion Primitives

In this section we describe how a library of feedback motion primitives Φ is constructed from the recorded set of data-points D offline and how it is applied during the online execution. Each data-point $d \in D$ is a tuple $(\mathbf{x}_b, \mathbf{u}_b, \mathbf{x}_{b+1})$ where $\mathbf{x}_b$ and $\mathbf{x}_{b+1}$ are from the system state space $\mathcal{X} \subseteq \mathbb{R}^{M\times 6}$ and $\mathbf{u}_b$ is from the system input space $\mathcal{U} \subseteq \mathbb{R}^{M\times 3}$.

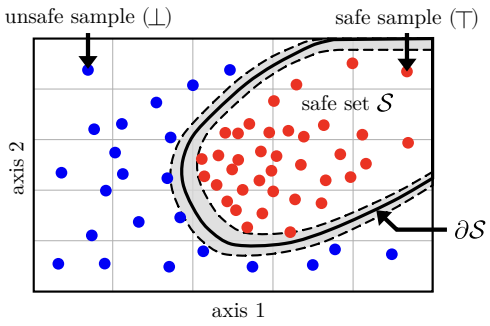


Fig. 3: Illustration of approximating the safe set $\mathcal{S}$ through positive (safe) and negative (unsafe) samples in red and blue respectively. The use of GPs allows us to provide a probabilistic estimate of the safe set $\mathcal{S}$ and its boundary $\partial\mathcal{S}$.

Algorithm 1: Offline construction of the motion primitive library

1 **Input**: Data-points D, Network N, set of inputs $\mathbf{U}^{ff}$;
2 **Output**: Feedback motion primitive library Φ;
3 $\Phi = \{\}$;
4 **for** u^{ff} *in* $\mathbf{U}^{ff}$ **do**
5 $\quad D' \leftarrow D$;
6 $\quad$ **while** D' *is not empty* **do**
7 $\quad\quad d \leftarrow pop(D')$;
8 $\quad\quad X \leftarrow d \oplus \sigma^x; \quad U \leftarrow u^{ff} \oplus \sigma^u$;
9 $\quad\quad \hat{f} \leftarrow compute\_local\_model(N, X, U)$;
10 $\quad\quad$ **for** d' *in* D' **do**
11 $\quad\quad\quad$ **if** $model\_is\_valid(\hat{f}, d')$ **then**
12 $\quad\quad\quad\quad X \leftarrow X \cup d' \oplus \sigma^x$;
13 $\quad\quad\quad\quad remove(d, D')$;
14 $\quad\quad\quad C \leftarrow synthesize\_feedback\_rule(\hat{f}, U)$;
15 $\quad\quad\quad R \leftarrow (U, X)$;
16 $\quad\quad\quad \Phi \leftarrow \Phi \cup \{(u^{ff}, \hat{f}, C, R)\}$;
17 $\quad$ **return** Φ

Each feedback motion primitive $\phi_i \in \Phi$ is executed during a fixed time interval $[T]$ and it is expressed as a tuple $(u^{ff}, \hat{f}, C, R)$ where $u^{ff} : [T] \rightarrow \mathcal{U}$ is a feed-forward input, $\hat{f} : \mathbf{x}_b^+ = A\mathbf{x}_b + B\mathbf{u}_b + c \oplus W$ is an affine estimation of f subject to bounded disturbance, and $C : X \rightarrow U$ is a feedback rule. $R : (U, X)$ is called the feedback motion primitive's region of validity where $U \subseteq \mathcal{U}$ and $X \subseteq \mathcal{X}$ are subsets of the input and state spaces respectively. Algorithm 1 outlines the construction of feedback motion primitives from a set of data-points D that is consisted of the states recorded during trial runs.

In Algorithm 1, we iterate over a finite set of inputs $U^{ff} \subset \mathbb{R}^{M\times 3}$ that correspond to the feed-forward part of the feedback motion primitives (line 4). For each input $u^{ff} \in U^{ff}$ we go through the data-points one by one. We define the *vicinity* of a data point d, as a hyper-interval $X \subset \mathbb{R}^{M\times 6}$ with fixed size $|\sigma^x|$, that is a design parameter. Similarly, the vicinity of the input u^{ff} is defined with the fixed size $|\sigma^u|$ (line 8). We compute a local model as an affine estimation $\hat{f}$ in the defined vicinity $X \times U$ by taking samples in this region from the neural network and solving the system identification problem as a linear programming problem (LP) (line 9). After constructing the local model for a data-point d, the remainder of the data-points in D' are checked for compliance with this model, i.e. if the affine estimation $\hat{f}$ holds at those points (line 11). Points that conform to the model will be removed from the data-set, while their corresponding vicinity will be added to the *region-of-validity* of the local model. We then synthesize a feedback rule C, again via formulation as an LP, to minimize the deviation from a prescribed state. As a result the refined input generated by the feedback motion primitive becomes $u^{ff} + C(\mathbf{x}^*_{b-1} - \mathbf{x}_{b-1})$, with $\mathbf{x}^*_{b-1}$ being the prescribed system state in the last block. The construction of feedback motion primitives continues until we have valid models for every data-point. We define the *default feedback motion primitive* as the one with the largest region of validity.

Motion Refinement During the execution time, a feed-forward input $u^{ff*} \in U^{ff}$ is selected by the MPC controller. The state input pair $(\mathbf{x}_b, u^{ff*})$ is then checked against the regions of validity and the refined input $u^{ff*} + C(\mathbf{x}^*_b - \mathbf{x}_b)$ is generated by the valid feedback motion primitive. When $(\mathbf{x}_b, u^{ff*})$ does not belong to any region of validity, we use the default feedback motion primitive to generate the input.

IV. EVALUATION

The goal of this work is to improve the baseline DDC (Section II-A) by increasing its safety with a GP-based approximation of safe sets, and its task execution efficiency by synthesizing feedback motion primitives. To explore the effect of the proposed method, we conduct the following experiments:

- In Sec. IV-B, we evaluate the approximated safe set by analyzing the classification accuracy of the Gaussian Process Classifier (GPC).
- In Sec. IV-C, we examine the improvement in tracking performance by incorporating the motion refinement feedback.
- In Sec. IV-D we inspect the improvement in the percentage of safe states for several weights in the cost function by considering the full architecture
- Concluding, in Sec. IV-E we demonstrate that the proposed approach is efficient in terms of execution time, as well as average task progress.

To illustrate these results we consider a box-pushing task along axis Y, as shown in Fig. 4. The default values for the parameters used for all experiments are reported in Tab. II.

A. Implementation Details

Dataset Collection: In the experiments, we use a YuMi-IRB 14000 collaborative robot manufactured by ABB, with an OptoForce 6axis Force/Torque sensor mounted on its wrist. To collect data, we command a pushing trajectory consisting of three pushes with the admittance controller presented in [32]. To alter the frictional properties of the task and create variation in the dynamics, we add loads $(275, 360, 525, 645\text{g})$ to the box and use different materials

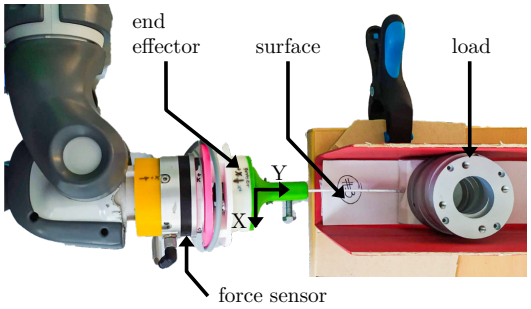


Fig. 4: Considered pushing task. The robot needs to push an object with different loads on various surfaces. The end-effector of the robot is equipped with a force sensor.

for the surface of the motion. The materials were chosen to cover a variety of different textures and frictional profiles. Surfaces and loads denoted as "Unknown" are used exclusively for testing. The materials used (IDs in parentheses), in ascending order of roughness, are the following:

- Unknown Surface 1 (U1): Plastic paper sleeve
- Surface 1 (1): Smooth crafting paper
- Surface 2 (2): Felt
- Surface 3 (3): Gouache paper
- Surface 4 (4): Cork
- Surface 5 (5): AP-100 Sandpaper
- Unknown Surface 2 (U2): AP-60 Sandpaper

Using surfaces 1-5 and loads of $0, 275, 370, 645$g, we gather 48 datasets of approximately 3000 datapoints, at 200Hz. Since we employ blocks of measurements, every block has $M = 10$ timesteps, resulting in datasets of 300 blocks each. The test set used for the experimental results in this section includes trials with known load (275g, also denoted as low load LL), known surfaces 1-5, an unknown load (525g, also denoted as high load HL) and the unknown surfaces U1 and U2.

Network Architecture and Training: The network modeling the dynamics in Eq. (2) is a fully connected network with 2 hidden layers with 30 neurons each, hyperbolic tangent activations and a dropout layer with probability $p = 0.3$ before the output layer. The training objective is the Mean Squared Error (MSE) for the prediction of the full state $\mathbf{x}_b$ for the next block and lasts 50 epochs on a 75-25 train-validation split, with learning rate $lr =$ 1e-05, weight decay $wd =$ 5e-05 and the Adam optimizer. The training hyper-parameters and the architecture were chosen through cross-validation. We utilize the dropout layer to avoid overfitting and to extract an estimate of the model's predictive uncertainty. For the validation set, the model had an MSE $= 0.027(\pm 0.0026)$.

Approximated Safe Set: In our example task, we want to avoid situations in which the robot is getting stuck while pushing. In these situations, the robot might get damaged if it continues to push with increasing forces, particularly if the object is immovable. In this regard, we consider a state as safe if the robot is able to continue its motion in the *subsequent* state:

$$\mathbf{x}_b \text{ is safe} \iff \exists \mathbf{u}_b : d(\mathbf{x}_b, f(\mathbf{x}_b, \mathbf{u}_b)) \geq l_t, \qquad (5)$$

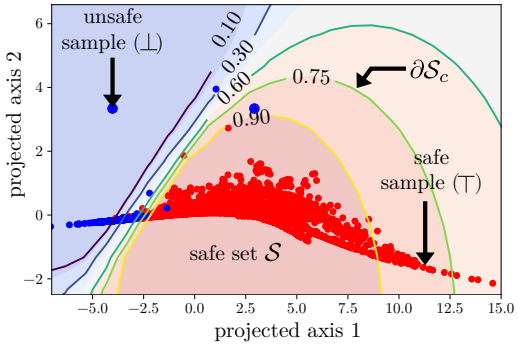


Fig. 5: Projection of the approximated safe set onto two dimensions using an Isomap manifold. The safe set boundary $\partial\mathcal{S}_c$ for different confidence scores $c \in \{0.1, 0.3, 0.6, 0.75, 0.9\}$ are shown in color. The examples of unsafe and safe states are shown as blue and red dots, respectively.

where $d(\mathbf{x}_b, f(\mathbf{x}_b, \mathbf{u}_b))$ returns the displacement between two states and l_t is a minimum displacement.

To approximate the safe set $\mathcal{S}$, we use the collected training data and partition it into examples of safe and unsafe transitions considering (5). In various experiments, we empirically determine the value of l_t so that the robot continues to move. Note that by choosing larger values for l_t, more states are classified as unsafe and $\mathcal{S}$ becomes smaller, resulting in more conservative classifications. We obtain dataset sizes of $|\mathcal{D}_{\text{safe}}| = 2533$ and $|\mathcal{D}_{\text{unsafe}}| = 1087$. We use the Gaussian Process Classifier (GPC) API of the Python library Scikit to approximate $\mathcal{S}$. We obtain the highest classification accuracy by only taking the relative positions within a state into account. Fig. 5 illustrates the resulting safe set. We use an Isomap manifold to project the 10-dimensional set to a two dimensional plane.

Model Predictive Controller: We solve Eq. (3) numerically with a random shooting method [3], [33], by uniformly sampling 5 feasible inputs in the range $[-10, 3]$, to act as $\mathbf{u}_b^*$ candidates. With the current task geometry, negative inputs move the robot towards the target, while the positive ones away from it. Based on the network's predictions for the next state (Eq. (2)) and the output of the GP safety classifier for this prediction, we evaluate the cost associated with each input. If the feasible input will lead the system to an unsafe state, we automatically assign it a large cost and effectively ignore it for this optimization round.

The cost function for the MPC in our experiments is:

$$\begin{aligned} C(\mathbf{x}_{b+k}, \mathbf{u}_{b+k}) = c_P \left(\mathbf{x}_{b+k}^{P,y} + x_b^{P,y} - \mathbf{p}_{target}\right)^2 \\ + c_F \left\|\mathbf{x}_{b+k}^{F,y}\right\|^2 + c_U \left\|\mathbf{u}_{b+k}\right\|^2, \end{aligned} \qquad (6)$$

where $\mathbf{x}^{P,y}$ and $\mathbf{x}^{F,y}$ are the predicted displacements and forces along the motion of axis respectively, $x_b^{P,y}$ is the current position of the end-effector that transforms the displacement to the future absolute position, $\mathbf{p}_{target}$ is the target position and c_P, c_F, c_U are positive constants that weigh the contribution of the individual cost terms. The first term encodes the desired position of the pushed object while the second and third terms, encourage solutions that lead to small predicted forces and have small magnitudes.

TABLE I: F_1-scores for the GPC for combinations of known and unknown loads and surfaces. The overall scores are 0.96 for Safe and 0.78 for Unsafe states, respectively.

Surfaces	Load	Safe	Unsafe
Known	Known	0.96	0.83
	Unknown	0.95	0.72
Unknown	Known	0.96	0.81
	Unknown	0.97	0.78

TABLE II: Default values of the parameters used in the experiments.

Dynamics Model	lr	1e-05
	wd	5e-05
	epochs	50
	M	10
Motion Primitives	σ^x	1e-05
	σ^y	4
	vicinity samples	1e+03
	MP range	1
Safe Sets	l_t	8e-05
	safety likelihood	0.9
MPC	p_{target}	-0.1
	c_P	1e+04
	c_F	5e-02
	c_U	5e-03

B. GPC Prediction Accuracy

We first evaluate the classification accuracy of GPC. For this experiment, we use all of the surfaces the network has trained on (known surfaces) and the two unseen ones (unknown surfaces), the load the network has encountered before (known load) and the new one (unknown load). We perform 3 iterations for every combination of surface and load. The F_1-scores for each class can be seen in Tab. I. The results indicate that GPC can accurately predict the sets of safe states, but has lower accuracy for the unsafe ones. Interestingly, the accuracy for the unknown surfaces and an unknown load is higher than the one for known surfaces and unknown load. This is potentially because surface U1 is a high friction surface and the force signals are sharp, making the decision easier. Additionally, surface U2 is a smooth surface with fewer stick-slip phenomena to overcome, leading to an overall higher score.

C. Deviation from Prescribed States

Next, we evaluate the effect of incorporating the feedback motion primitives in the MPC loop and compare it with the baseline approach from [32], denoted as `Vanilla` in the following. In this experiment, we execute pushing trials on all the available surfaces and loads. For every combination of surface and load, we repeat the trial 3 times, resulting in 84 trials, 42 with feedback motion primitives and 42 for `Vanilla`. We observe that the input adjustment through the motion primitive feedback reduces the displacement error from an average of 0.28 to 0.20, corresponding to an error reduction of 30%. The error comparison is shown in Fig. 6.

D. Execution Safety

To examine the overall safety of the proposed approach, we compare the baseline that uses the network and random

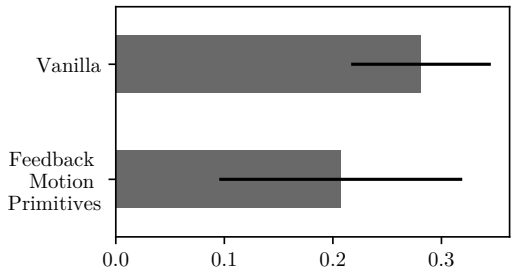
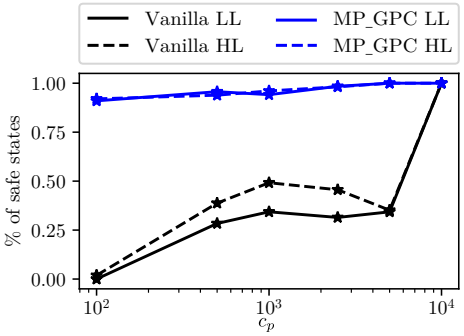
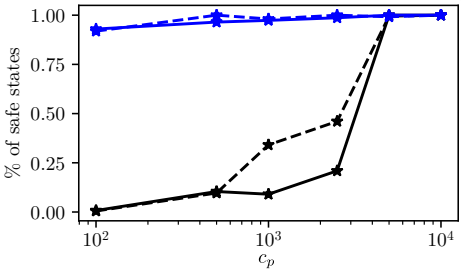


Fig. 6: Improvement of the deviation from the prescribed states (relative displacements, in mm) by incorporating the feedback motion primitives to the `Vanilla` baseline.



(a) Executions with different c_P on known surface with low (LL) and high (HL) loads.



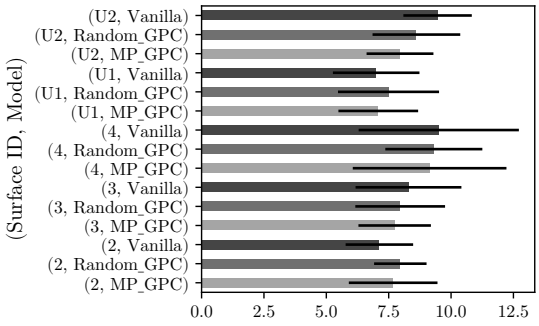
(b) Executions with different c_P on unknown surface with low (LL) and high (HL) loads.

Fig. 7: Percentage of safe states during executions with different c_P weights on the MPC cost function for a known (top) and unknown (bottom) surface. The proposed approach `GP-MPC` keeps the system in the safe set for 90% of the states, regardless of the load or the surface. In contrast, `Vanilla`, reaches the same performance only for $c_P \in \{5000, 10000\}$.

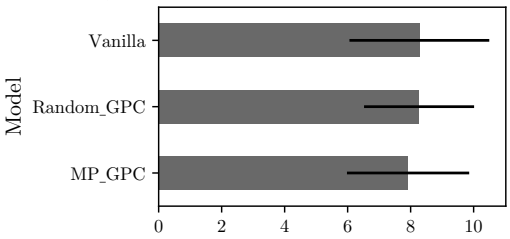
shooting `Vanilla` and our integrated method `MP-GPC` in terms of the percentage of safe states. We look into the percentage of safe states over the task duration, for different values of the parameter c_P in Eq. (6). The smaller c_P is, the less dominating the first term of the cost function will be and the controller might opt for solutions that prioritize smaller predicted forces or control input norm, instead of higher progress toward the goal. Hence, smaller c_P results in a less aggressive tuning of the MPC.

The percentage of safe states are presented in Fig. 7 for the known surface 3 (Fig. 7a) and the unknown surface U2 (Fig. 7b) with the known Low Load (LL = 275g) and the unknown High Load (HL = 525g). The goal for every trial is to reach the target position p_{target}. We observe that the percentage of safe states evolves as we encourage behaviors of increasing aggressiveness from the MPC.

In terms of failed executions, with `GP-MPC` the only

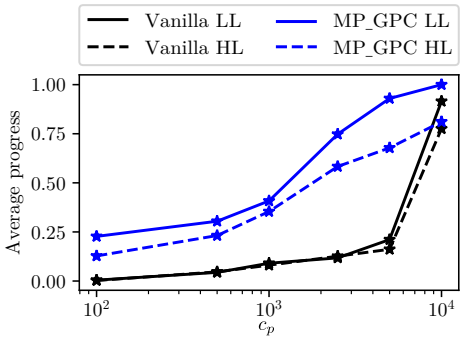


(a) Average task ET (sec) for all the surfaces individually.

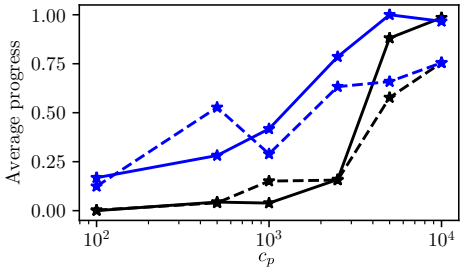


(b) Average ET (sec) for every baseline across all the surfaces and loads.

Fig. 8: Execution times (ET, in sec) for the three baselines per surface (top) and on average over surfaces and loads (bottom).



(a) Different c_P on known surface with low (LL) and high (HL) loads.



(b) Different c_P on unknown surface with low (LL) and high (HL) loads.

Fig. 9: Averaged task progress over the task duration, during executions with different c_P weights on the cost function for a known (top) and unknown (bottom) surface. The proposed approach `MP-GPC` outperforms the baseline `Vanilla` for all c_P values and combinations of surfaces and loads.

failed trials are observed for $c_P = 100$ for both loads and surfaces, as the control inputs are not adequate to overcome friction within the allotted time. Additionally, for the unknown surface and $c_P \in \{100, 500, 1000\}$, the GPC terminated the trial preemptively when all potential inputs within the acceptable range were deemed unsafe. In contrast, the `Vanilla` baseline is not able to complete the task for any value smaller than 5000.

E. Execution Efficiency

Finally, to examine the method's efficiency in terms of task execution, we evaluate how the execution time is affected by the addition of the corrective feedback controller and the safety layer of GPC. For the trials, we use the known surfaces 2-4, the unknown surfaces U1, U2, the known LL, the unknown HL and the objective is to reach a target position p_{target}.

The architectures we compare are the baseline approach (`Vanilla`), the baseline that also includes GPC (`Random-GPC`) and finally the proposed architecture that includes the corrective feedback motion primitives (`MP-GPC`). For the two architectures that perform safety checks (`Random-GPC`,`MP-GPC`), if a feasible input is deemed unsafe, it is not included in the MPC's optimization round. If all of the feasible inputs are unsafe, the robot is commanded to retract its end-effector for that control round.

From Fig. 8, we can see that despite the additional safety checks, the proposed approach reaches the target faster. Specifically, the addition of the safety check alone for `Random-GPC` slightly decreases both the mean execution time, as well as the standard deviation compared to `Vanilla`. This is a result of rejecting the inputs that can not overcome friction and instead focusing on the ones that ensure motion continuation. The average execution time is further decreased by including the motion primitives that minimize deviation from the prescribed states, allowing `MP-GPC` to reach the target faster for almost all of the surfaces, except surface 2.

Separate to the execution time, we also examine the average task progress over the task duration, for different values of the weight c_P in Eq. (6) and present the results in Fig. 9. For this experiment, we consider the surfaces 3, U2 and the same loads as before ($275, 525$g). For the pushing constant, we choose values $c_P \in \{100, 500, 1000, 2500, 5000, 10000\}$ and the goal is to reach p_{target}. A trial is considered unsuccessful and terminated, if the robot has not reached the target within one minute. In the case of `MP-GPC`, if at any control iteration, all potential inputs lead to unsafe states, we also terminate the trial to avoid excessive effort.

To compute the average progress, we evaluate how close to the target the box was pushed and average it over how long it took to reach the final position. We observe that the proposed method outperforms the baseline for every combination. Specifically, for the known surface in Fig. 9a the baseline `Vanilla` does not exceed 0.25 with a weight smaller than $c_P = 5000$, while `MP-GPC` displays the same progress for $c_P = 100$ and reaches almost 0.7 progress for the heavy load using $c_P = 5000$. For the unknown surface in Fig. 9b, `Vanilla` has more comparable performance for the different c_P values but still under-performs for every value and load. The executions with $c_P \in \{100, 500, 1000\}$ and high load (HL) on the unknown surface (Fig. 9b), terminated preemptively. The GPC interrupts the execution when none of the inputs in the accepted range could lead to a safe state.

V. CONCLUSIONS

This paper addressed the safety of learning-based DDC for contact-rich manipulation tasks. We showed that safe sets can be approximated and integrated into the DDC framework. In addition, we synthesized feedback motion primitives to ensure that the closed-loop system remains in the safe set, despite deviations from the prescribed states. The resulting controller performed better and exhibited safer behaviors. The experiments demonstrated that our method can predict unsafe states (Sec.IV-B) for a combination of surfaces and loads. By incorporating the feedback motion primitives, we effectively reduce the deviation from the prescribed states (Sec.IV-C). Our enhanced controller also exhibited improved performance in terms of execution time and task progress (Sec. IV-E), when compared to the baseline. Lastly, by comparing the overall safety and how it is affected by encouraging the controller to find solutions of different aggressiveness, we showed that the proposed method succeeds in keeping the system in the safe states regardless of the weight, surface or load used Sec.(IV-D).

In the future, we aim to explore different safety definitions and how they affect the DDC performance, as well as their application to tasks with higher dimensional dynamics.

REFERENCES

[1] O. Kroemer, S. Niekum, and G. Konidaris, "A review of robot learning for manipulation: Challenges, representations, and algorithms," *Journal of Machine Learning Research*, vol. 22, no. 30, pp. 1–82, 2021.

[2] S. A. Khader, H. Yin, P. Falco, and D. Kragic, "Data-efficient model learning and prediction for contact-rich manipulation tasks," *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4321–4328, 2020.

[3] A. Nagabandi, G. Kahn, R. S. Fearing, and S. Levine, "Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning," in *Proc. of the IEEE Int. Conf. on Robotics and Automation*, 2018, pp. 7559–7566.

[4] M. T. Mason, "Mechanics and planning of manipulator pushing operations," *The Int. J. of Robotics Research*, vol. 5, no. 3, pp. 53–71, 1986.

[5] M. Bauza and A. Rodriguez, "A probabilistic data-driven model for planar pushing," in *Proc. of the IEEE Int. Conf. on Robotics and Automation*, 2017, pp. 3008–3015.

[6] J. Li, W. S. Lee, and D. Hsu, "Push-net: Deep planar pushing for objects with unknown physical properties," in *Robotics: Science and Systems*, 2018.

[7] C. C. Beltran-Hernandez, D. Petit, I. G. Ramirez-Alpizar, T. Nishi, S. Kikuchi, T. Matsubara, and K. Harada, "Learning force control for contact-rich manipulation tasks with rigid position-controlled robots," *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5709–5716, 2020.

[8] S. Pathak, L. Pulina, G. Metta, and A. Tacchella, "Ensuring safety of policies learned by reinforcement: Reaching objects in the presence of obstacles with the iCub," in *Proc. of the IEEE/RSJ Int. Conf. on Intel. Robots and Systems*, 2013, pp. 170–175.

[9] S. H. Huang, M. Zambelli, J. Kay, M. F. Martins, Y. Tassa, P. M. Pilarski, and R. Hadsell, "Learning gentle object manipulation with curiosity-driven deep reinforcement learning," *arXiv preprint arXiv:1903.08542*, 2019.

[10] R. Martín-Martín, M. A. Lee, R. Gardner, S. Savarese, J. Bohg, and A. Garg, "Variable impedance control in end-effector space: An action space for reinforcement learning in contact-rich tasks," in *Proc. of the IEEE/RSJ Int. Conf. on Intel. Robots and Systems*, 2019, pp. 1010–1017.

[11] J. Yamada, Y. Lee, G. Salhotra, K. Pertsch, M. Pflueger, G. S. Sukhatme, J. J. Lim, and P. Englert, "Motion planner augmented reinforcement learning for obstructed environments," in *Conf. on Robot Learning*, 2020.

[12] C.-Y. Kuo, A. Schaarschmidt, Y. Cui, T. Asfour, and T. Matsubara, "Uncertainty-aware contact-safe model-based reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3918–3925, 2021.

[13] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. of AAAI Conf. on Artificial Intelligence*, 2018.

[14] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *Proc. of the European Control Conference*, 2019, pp. 3420–3431.

[15] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2016.

[16] L. Lindemann and D. V. Dimarogonas, "Control barrier functions for signal temporal logic tasks," *IEEE Control Systems Letters*, vol. 3, no. 1, pp. 96–101, 2018.

[17] W. S. Cortez and D. V. Dimarogonas, "Correct-by-design control barrier functions for euler-lagrange systems with input constraints," in *Proc. of the American Control Conference*, 2020, pp. 950–955.

[18] K. Berntorp, A. Weiss, C. Danielson, and S. Di Cairano, "Automated driving: Safe motion planning using positively invariant sets," in *Proc. of the IEEE Int. Conf. on Intelligent Transportation Systems*, 2017, pp. 2247–2252.

[19] D. Althoff, M. Althoff, and S. Scherer, "Online safety verification of trajectories for unmanned flight with offline computed robust invariant sets," in *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, 2015, pp. 3470–3477.

[20] M. Jalalmaab, B. Fidan, S. Jeon, and P. Falcone, "Guaranteeing persistent feasibility of model predictive motion planning for autonomous vehicles," in *Proc. of the IEEE Intelligent Vehicles Symposium*, 2017, pp. 843–848.

[21] C. Pek and M. Althoff, "Fail-safe motion planning for online verification of autonomous vehicles using convex optimization," *IEEE Transactions on Robotics*, pp. 1–17, 2020.

[22] Y. Sui, A. Gotovos, J. Burdick, and A. Krause, "Safe exploration for optimization with gaussian processes," in *Int. Conf. on Machine Learning*, 2015, pp. 997–1005.

[23] C. E. Rasmussen, "Gaussian processes in machine learning," in *Summer School on Machine Learning*. Springer, 2003, pp. 63–71.

[24] F. Berkenkamp, M. Turchetta, A. Schoellig, and A. Krause, "Safe model-based reinforcement learning with stability guarantees," in *Advances in neural information processing systems*, 2017, pp. 908–918.

[25] P. Englert and M. Toussaint, "Combined optimization and reinforcement learning for manipulation skills." in *Robotics: Science and systems*, 2016.

[26] J. Nubert, J. Köhler, V. Berenz, F. Allgöwer, and S. Trimpe, "Safe and fast tracking on a robot manipulator: Robust mpc and neural network control," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3050–3057, 2020.

[27] A. Carron, E. Arcari, M. Wermelinger, L. Hewing, M. Hutter, and M. N. Zeilinger, "Data-driven model predictive control for trajectory tracking with a robotic arm," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3758–3765, 2019.

[28] T. Westenbroek, D. Fridovich-Keil, E. Mazumdar, S. Arora, V. Prabhu, S. S. Sastry, and C. J. Tomlin, "Feedback linearization for uncertain systems via reinforcement learning," in *Proc. of the IEEE Int. Conf. on Robotics and Automation*, 2020, pp. 1364–1371.

[29] B. Schürmann and M. Althoff, "Guaranteeing constraints of disturbed nonlinear systems using set-based optimal control in generator space," *IFAC Proceedings*, vol. 50, no. 1, pp. 11 515–11 522, 2017.

[30] A. Majumdar and R. Tedrake, "Funnel libraries for real-time robust feedback motion planning," *The Int. J. of Robotics Research*, vol. 36, no. 8, pp. 947–982, 2017.

[31] P. Tajvar, A. Varava, D. Kragic, and J. Tumova, "Robust motion planning for non-holonomic robots with planar geometric constraints," in *Symposium on Robotics Research*, 2019.

[32] I. Mitsioni, Y. Karayiannidis, and D. Kragic, "Modelling and learning dynamics for robotic food-cutting," *arXiv preprint arXiv:2003.09179*, 2020.

[33] A. Rao, "A survey of numerical methods for optimal control," *Advances in the Astronautical Sciences*, vol. 135, 01 2010.